



# TRƯỜNG ĐIỆN – ĐIỆN TỬ

KHOA ĐIỆN TỬ

## KỸ THUẬT PHẦN MỀM ỨNG DỤNG

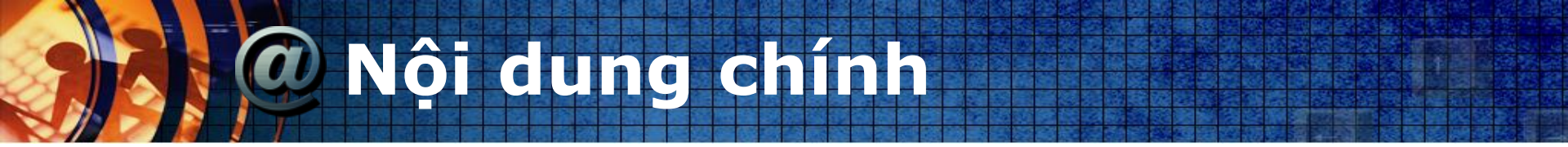
### Chương 2

### Các pha trong phát triển phần mềm

#### *Phần 6*

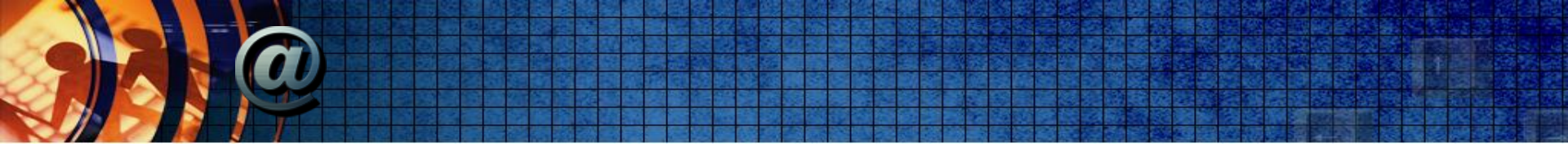
#### *Thiết kế hệ thống – Thiết kế cơ sở dữ liệu*

|         |         |       |         |         |        |         |    |
|---------|---------|-------|---------|---------|--------|---------|----|
| 3898043 | 5660163 | 63758 | 6752245 | 7196671 | 154963 | 2867239 | 17 |
| OAS     | ODS     | MAV   | WAV     | IDS     | VUIL   | KAS     | 1  |



# @ Nội dung chính

- Tổng quan về Mô hình dữ liệu quan hệ
- Chuẩn hóa dữ liệu
- Chuyển từ mô hình thực thể liên kết sang mô hình quan hệ



# Tổng quan về Mô hình dữ liệu quan hệ

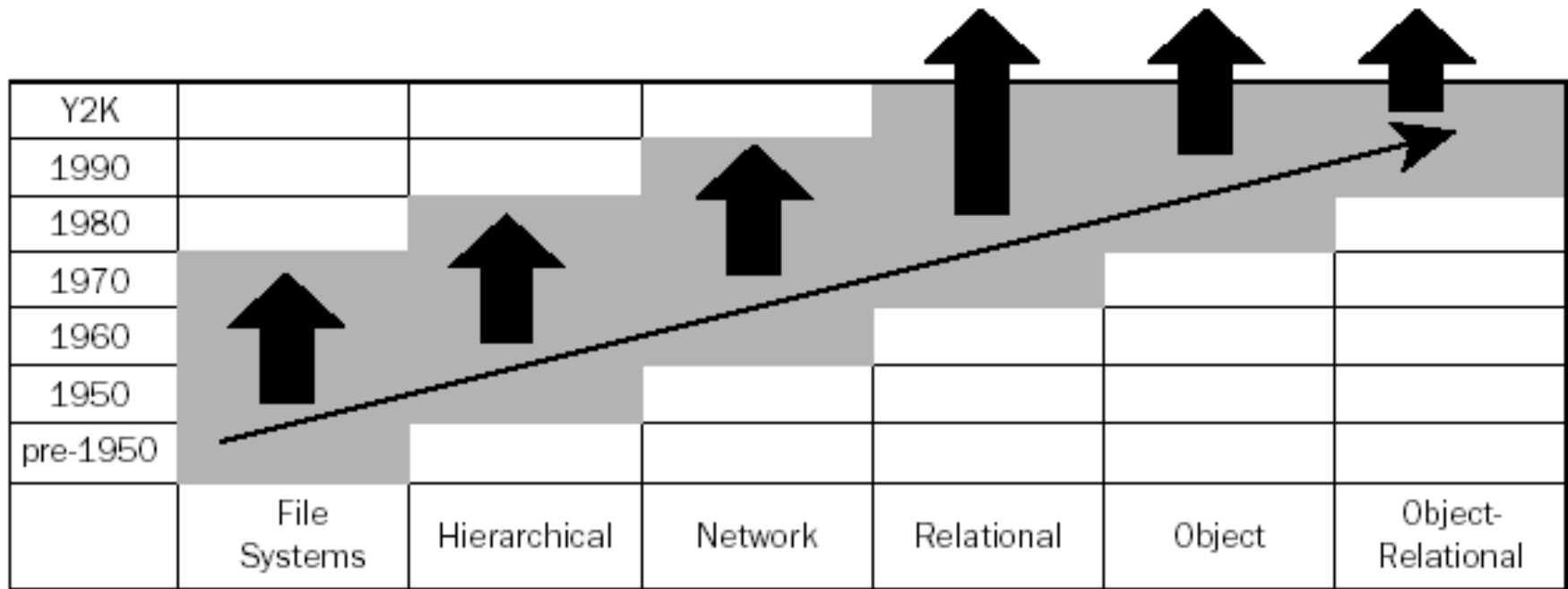


# Mô hình dữ liệu là gì?

- Mô hình dữ liệu (Data Model) là cách ký hiệu để mô tả dữ liệu hay thông tin. Nói chung, nó gồm có ba phần:
  - Cấu trúc của dữ liệu (Structure of the data)
  - Các thao tác trên dữ liệu (Operations on data)
  - Các ràng buộc (Constraints)



# Tóm tắt lịch sử của các Mô hình dữ liệu



The evolution of database modeling techniques.



## Mô hình dữ liệu quan hệ (Relational Data Model)

- Quan hệ (relation) là một bảng hai chiều:
  - Các cột gọi là các thuộc tính (attribute)
  - Các hàng gọi là các bản ghi (record) hay bộ (tuple)
- Ví dụ: Quan hệ Customers (bảng Customers)
  - Mỗi hàng biểu diễn cho một CUSTOMER
  - Mỗi cột biểu diễn cho một thuộc tính của CUSTOMER và được gọi là “thuộc tính”

| Customer ID | Tax ID      | Name       | Address          |
|-------------|-------------|------------|------------------|
| 1234567890  | 555-5512222 | Munmun     | 323 Broadway     |
| 2223344556  | 555-5523232 | Vijeta     | 1200 Main Street |
| 3334445563  | 555-5533323 | Ekta       | 871 1st Street   |
| 4232342432  | 555-5325523 | E. F. Codd | 123 It Way       |





# Mô hình dữ liệu quan hệ (Relational Data Model)

- **Lược đồ (Schema)**
  - Lược đồ của một quan hệ là thông tin về cấu trúc của quan hệ đó
  - Nó bao gồm tên và tập các thuộc tính của quan hệ đó.
  - Ví dụ: Lược đồ của quan hệ Customers ở trên:
    - Customers(Customers ID, tax ID, name, address)
- **Bộ và thành phần**
  - Một hàng của một quan hệ được gọi là một bộ (tuple) hay bản ghi (record)
  - Một giá trị của một thuộc tính trong một bộ được gọi là một thành phần (component)

| Customer ID | Tax ID      | Name       | Address          |
|-------------|-------------|------------|------------------|
| 1234567890  | 555-5512222 | Munmun     | 323 Broadway     |
| 2223344556  | 555-5523232 | Vijeta     | 1200 Main Street |
| 3334445563  | 555-5533323 | Ekta       | 871 1st Street   |
| 4232342432  | 555-5325523 | E. F. Codd | 123 It Way       |



# Mô hình dữ liệu quan hệ (Relational Data Model)

- Miền giá trị (Domains)
  - Miền giá trị của một thuộc tính là tập tất cả các giá trị mà thuộc tính đó có thể nhận
  - Nên khái niệm miền giá trị tương đương với khái niệm kiểu dữ liệu (data type)
  - Ở mức cài đặt trong một hệ quản trị CSDL, thì miền giá trị của các thuộc tính của một quan hệ cần phải chọn sao cho nằm trong số các kiểu dữ liệu mà hệ QT CSDL đó hỗ trợ





# Mô hình dữ liệu quan hệ (Relational Data Model)

- Biểu diễn tương đương của quan hệ
  - Quan hệ là tập hợp của các bộ (sets of tuples), chứ không phải danh sách của các bộ (lists of tuples)
  - Do vậy, thứ tự của các bộ trong quan hệ không quan trọng, mà quan trọng là nội dung của các bộ. Đồng thời nếu có 2 bộ có giá trị bằng nhau trên mọi thuộc tính thì coi như 1 bộ bị thừa
- Thể hiện của quan hệ
  - Thể hiện của quan hệ (instance of a relation) là tập các bộ của quan hệ đó tại một thời điểm
  - Thể hiện của một quan hệ phản ánh nội dung của quan hệ đó, và nó thay đổi khi có các thao tác cập nhật nội dung quan hệ như thêm, sửa hay xóa bộ.



# Mô hình dữ liệu quan hệ (Relational Data Model)

- Khóa của quan hệ
  - là một hoặc một tập hợp các thuộc tính của  $r$ , sao cho với mọi thể hiện của  $r$ , thì không tồn tại 2 bộ nào mà giá trị của các thành phần bằng nhau trên khóa.
  - Hay nói cách khác, giá trị của các thành phần trên Khóa của mỗi bộ trong mọi thể hiện của quan hệ  $r$  là duy nhất.
- Có hai loại Khóa của quan hệ
  - Khóa của một thể hiện của quan hệ
  - Khóa của lược đồ của quan hệ: là khóa của mọi thể hiện của quan hệ



# Mô hình dữ liệu quan hệ (Relational Data Model)

- Khóa của quan hệ

VD: với thể hiện s1 của quan hệ Student sau:

| ID_SV    | Name          | Address    | Class |
|----------|---------------|------------|-------|
| 20091111 | Nguyễn Văn A  | Hà nội     | ĐT1   |
| 20091112 | Nguyễn Hồng B | Quảng ninh | ĐT2   |
| 20091113 | Trần Lê C     | Hà tây     | ĐT1   |
| 20091114 | Võ Thị D      | Bắc giang  | ĐT2   |

Có các Khóa là:

- K1 = ID\_SV
- K2 = Name
- K3 = Address



# Mô hình dữ liệu quan hệ (Relational Data Model)

- Khóa của quan hệ

VD: nhưng với thể hiện s2 của quan hệ Student sau:

| ID_SV    | Name                | Address       | Class |
|----------|---------------------|---------------|-------|
| 20091111 | <b>Nguyễn Văn A</b> | <b>Hà nội</b> | ĐT1   |
| 20091112 | Nguyễn Hồng B       | Quảng ninh    | ĐT2   |
| 20091113 | <b>Nguyễn Văn A</b> | Hà tây        | ĐT1   |
| 20091114 | Võ Thị D            | <b>Hà nội</b> | ĐT2   |

thì chỉ còn một Khóa là:  $K1 = ID\_SV$

=>  $K1 = ID\_SV$  là Khóa của lược đồ Sinh viên,

Còn  $K2$  và  $K3$  chỉ là Khóa của thể hiện s1



# Mô hình dữ liệu quan hệ (Relational Data Model)

- Khóa của quan hệ
  - Tính chất:
    - Khóa của thể hiện thì chưa chắc đã là Khóa của lược đồ
    - Còn Khóa của lược đồ thì cũng là Khóa của mọi thể hiện của quan hệ
    - => Khóa của lược đồ không phụ thuộc vào nội dung của quan hệ, còn khóa của thể hiện thì phụ thuộc





# @ Cơ sở dữ liệu quan hệ

- CSDL quan hệ (Relational Database): Là tập hợp các quan hệ và các ràng buộc giữa chúng, mà có liên quan đến một hệ thống thông tin (phần mềm) nào đó





# Các bất thường trong một quan hệ

- Quan sát quan hệ Student

|   | ID      | Name         | Class | Department  | Subject            | Mark |
|---|---------|--------------|-------|-------------|--------------------|------|
| 1 | E1-001  | Nguyen Van A | E1    | Electronics | Electronic Circuit | 8    |
| 2 | E1-001  | Nguyen Van A | E1    | Electronics | Digital Technique  | 7    |
| 3 | E1-002  | Tran Thi B   | E1    | Electronics | Digital Technique  | 9    |
| 4 | E1-002  | Tran Thi B   | E1    | Electronics | Electronic Circuit | 8    |
| 5 | E2-001  | Nguyen Ho C  | E2    | Electronics | Digital Technique  | 6    |
| 6 | IT1-001 | Tran Thi B   | IT1   | IT          | Electronic Circuit | 10   |
| 7 | IT1-002 | Le Van D     | IT1   | IT          | Digital Technique  | 8    |

- Dư thừa (Redundancy)
- Bất thường khi cập nhật (Update Anomalie)
- Bất thường khi bổ sung (Insertion Anomalie)
- Bất thường khi xóa (Deletion Anomalie)



# Các bất thường trong một quan hệ

- Hậu quả của các bất thường trên
  - Sự không nhất quán dữ liệu có nguy cơ rất cao
  - Tồn chỗ lưu trữ do dư thừa
  - Các thao tác cơ bản trên CSDL không hiệu quả
- Nguyên nhân của các bất thường trên
  - => Khái niệm **Phụ thuộc hàm** có thể giúp chúng ta hiểu nguyên nhân, cũng như giúp tìm ra giải pháp cho vấn đề trên.



# Phụ thuộc hàm

- Định nghĩa về PTH (Functional Dependency)
  - Cho lược đồ quan hệ  $R(A_1, A_2, \dots, A_n)$ , và  $A = A_1 \cup A_2 \cup \dots \cup A_n$ , và 2 tập các thuộc tính  $X$  và  $Y \subseteq A$ . Chúng ta nói rằng:
    - $X \rightarrow Y$  ( $X$  xác định hàm  $Y$ , hay  $Y$  phụ thuộc hàm vào  $X$ ), nếu  $\forall$  thể hiện  $r$  của  $R$ , thì không tồn tại 2 bộ  $t_1, t_2$ , sao cho  $t_1[X] = t_2[X]$  và  $t_1[Y] \neq t_2[Y]$
    - Nói cách khác: tập các thuộc tính  $Y$  phụ thuộc hàm vào tập các thuộc tính  $X$  khi và chỉ khi ứng với mỗi bộ giá trị của  $X$  trong  $R$  chỉ xác định đúng một giá trị của  $Y$  tại mọi thời điểm
  - $X$  được gọi là Quyết định (determinant) của PTH.
- Một số PTH từ bảng Student:
  - $ID \rightarrow Name$ ;
  - $ID \rightarrow Class$ ;
  - $Class \rightarrow Department$ ;
  - $(ID, Subject) \rightarrow Mark$



# @ Phụ thuộc hàm

- Ý nghĩa của một PTH  $X \rightarrow Y$  trong lược đồ R:  $\forall$  thể hiện r của R thì:
  - Hoặc không có bất kỳ 2 bộ t1, t2 nào, sao cho  $t1[X]=t2[X]$
  - Hoặc nếu tồn tại 2 bộ t1, t2 mà  $t1[X]=t2[X]$ , thì  $t1[Y]=t2[Y]$
  - Trong cả hai trường hợp,  $\forall$  bộ t, với mỗi giá trị của t[X] thì chỉ có một giá trị của t[Y], do đó nếu biết trước giá trị t[X], thì có thể xác định giá trị t[Y].



# @ Phụ thuộc hàm

- PTH đến từ đâu?
  - Nó xuất phát các **quy tắc nghiệp vụ** (hay logic nghiệp vụ) của cơ sở dữ liệu. Các quy tắc nghiệp vụ này sẽ quy định các phụ thuộc giữa các thuộc tính, và từ đó sẽ tạo ra các phụ thuộc hàm.
- Ví dụ: một số quy tắc nghiệp vụ trong lược đồ Student:
  - r1: Mỗi sinh viên có một ID riêng, duy nhất.
  - r2: Mỗi sinh viên chỉ có duy nhất 1 tên và chỉ thuộc về một lớp.
    - Từ r1 và r2, ta có các PTH sau:  
 $ID \rightarrow Name; ID \rightarrow Class$
  - r3: Với mỗi môn học, mỗi sinh viên chỉ thi một lần
    - Từ r3, ta có PTH:  $(ID, Subject) \rightarrow Mark;$





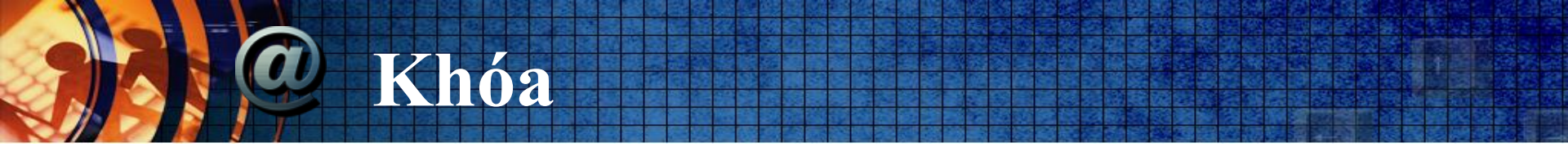
# Phụ thuộc hàm

- Hệ tiên đề Amstrong
  - Tính phản xạ:  $\forall Y \subseteq X$  thì  $X \rightarrow Y$  (PTH tầm thường)
  - Tính tăng trưởng:  $\forall X, Y, Z$ , nếu  $X \rightarrow Y$  thì  $XZ \rightarrow YZ$  ( $XZ = X \cup Z$ )
  - Tính bắc cầu:  $\forall X, Y, Z$ , nếu  $X \rightarrow Y$  và  $Y \rightarrow Z$  thì  $X \rightarrow Z$
- Các hệ quả:
  - Tính hợp : nếu  $X \rightarrow Y$  và  $X \rightarrow Z$  thì  $X \rightarrow YZ$
  - Tính tách: nếu  $X \rightarrow Y$  và  $Z \subseteq Y$  thì  $X \rightarrow Z$
  - Tính tựa bắc cầu: nếu  $X \rightarrow Y$  và  $YW \rightarrow Z$  thì  $XW \rightarrow Z$



# @ Phụ thuộc hàm

- Một số loại PTH đặc biệt
  - **PTH bộ phận** và **PTH đầy đủ**:  $X \rightarrow Y$  được gọi là PTH bộ phận nếu  $\exists X' \subset X$  sao cho  $X' \rightarrow Y$ . Trái lại, nếu không tồn tại  $X'$  như trên thì PTH này là đầy đủ.
  - **PTH bắc cầu** và **PTH trực tiếp**:  $X \rightarrow Y$  được gọi là PTH bắc cầu (hay gián tiếp) nếu  $\exists Z \not\subset XY$  sao cho  $X \rightarrow Z$  và  $Z \rightarrow Y$ . Trái lại, nếu không tồn tại  $Z$  như trên thì PTH này là trực tiếp.
  - Lưu ý: Sau đây, khi nói phụ thuộc hàm có nghĩa đó là phụ thuộc hàm đầy đủ và trực tiếp



- Có một số loại khóa như sau:
  - Siêu khóa (Super Keys).
  - Khóa ứng viên (Candidate Keys).
  - Khóa chính (Primary Keys).
  - Khóa ngoại.



- Siêu khóa

- Là một hay một tập các cột xác định duy nhất một hàng trong bảng
- SK là siêu khóa của R khi với mọi  $t_1, t_2$  thuộc bất kỳ thể hiện  $r$  của R ta luôn có  $t_1[SK] \neq t_2[SK]$
- SK là siêu khóa của R  $\Leftrightarrow$  SK xác định hàm mọi thuộc tính của R ( $SK \rightarrow R$ ).

Ví dụ: Cho lược đồ **DMSV**(MaSV, SoCMTND, Ho\_ten\_sv, Ngay\_Sinh, Dia\_chi, Que\_quan).

Hãy xác định các siêu khóa trong lược đồ trên.



# Khóa

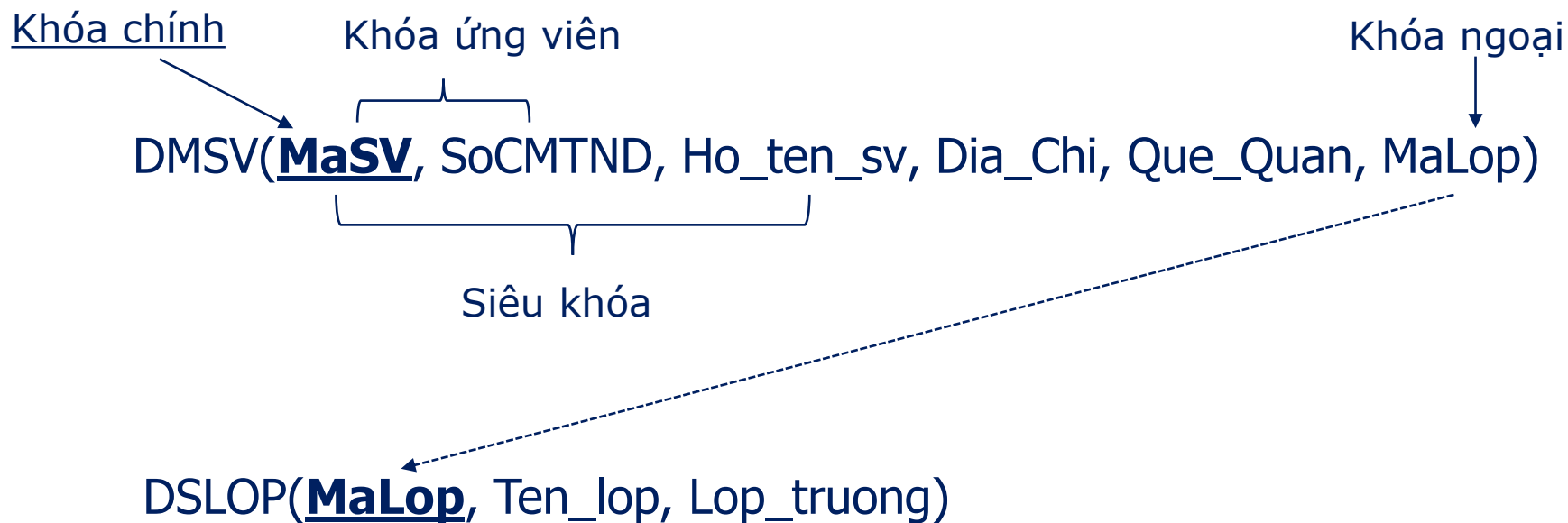
- Khóa ứng viên (khóa ứng cử): Là một tập con của siêu khóa (cũng là một siêu khóa), nhưng không thể thu lại thành một siêu khóa khác  
Ví dụ ta có tập các khóa ứng viên  $\{ (MaSV), (SoCMTND) \}$ .
- Khóa chính: Là một khóa được lựa chọn một cách tùy ý từ tập các khóa ứng viên, và được sử dụng trong các chỉ mục của bảng đó.
- Khóa ngoại: Một thuộc tính của quan hệ  $R1$  được gọi là một khóa ngoại nếu nó không phải là khóa chính của quan hệ  $R1$  nhưng các giá trị của nó là các giá trị của khóa chính trong một quan hệ  $R2$  nào đó
- Khóa ngoại lai dùng để chứa dữ liệu dùng cho việc liên kết với các bảng khác trong CSDL.





# Khóa

## Ví dụ về các khóa





# Khóa

- Thuộc tính khóa (Prime attribute): một thuộc tính A là thuộc tính khóa nếu nó tham gia vào thành phần của khóa chính hoặc khóa ứng cử. Trái lại, thì nó là thuộc tính không khóa (hay thuộc tính mô tả)



# Nguyên nhân gây ra các bất thường

|   | <b><u>ID</u></b> | <b>Name</b>  | <b>Class</b> | <b>Department</b> | <b><u>Subject</u></b> | <b>Mark</b> |
|---|------------------|--------------|--------------|-------------------|-----------------------|-------------|
| 1 | E1-001           | Nguyen Van A | E1           | Electronics       | Electronic Circuit    | 8           |
| 2 | E1-001           | Nguyen Van A | E1           | Electronics       | Digital Technique     | 7           |
| 3 | E1-002           | Tran Thi B   | E1           | Electronics       | Digital Technique     | 9           |
| 4 | E1-002           | Tran Thi B   | E1           | Electronics       | Electronic Circuit    | 8           |
| 5 | E2-001           | Nguyen Ho C  | E2           | Electronics       | Digital Technique     | 6           |
| 6 | IT1-001          | Tran Thi B   | IT1          | IT                | Electronic Circuit    | 10          |
| 7 | IT1-002          | Le Van D     | IT1          | IT                | Digital Technique     | 8           |



## Nguyên nhân gây ra các bất thường

- Khóa chính của quan hệ Student:
  - $K = (ID, Subject)$
- Nhận xét: các thuộc tính dư thừa là các thuộc tính phụ thuộc hàm bộ phận hoặc bắc cầu vào Khóa chính, ví dụ như:
  - $(ID, Subject) \rightarrow Name, Class, Department$ ; PTH bộ phận vì  
 $ID \rightarrow Name, ID \rightarrow Class, ID \rightarrow Department$
  - $Class \rightarrow Department$



## Nguyên nhân gây ra các bất thường

- Là sự tồn tại của các phụ thuộc bộ phận hoặc bắc cầu vào khóa chính
- Giải pháp: Cần loại bỏ các loại PTH này trong quan hệ, nhưng làm thế nào?

**→ Phép tách lược đồ!**





# Phép tách

- Định nghĩa: cho một LĐQH R. Một phép tách R là sự thay thế nó bằng các LĐ con (sub-schema)  $R_1, R_2, \dots, R_n$  sao cho  $R = R_1 \cup R_2 \cup \dots \cup R_n$ . Ký hiệu phép tách là:  $\rho(R) = (R_1, R_2, \dots, R_n)$  ( $\rho$  called rho)
- Một số tính chất muốn có của phép tách:
  - Tách nối không mất thông tin (Lossless-join decomposition)
  - Bảo toàn các PTH (Preservation of FDs)



## Ví dụ: tách bảng *Student* thành 3 bảng S1, S2 và S3

S1

| <u>ID</u> | Name         | Class |
|-----------|--------------|-------|
| E1-001    | Nguyen Van A | E1    |
| E1-002    | Tran Thi B   | E1    |
| E2-001    | Nguyen Ho C  | E2    |
| IT1-001   | Tran Thi B   | IT1   |
| IT1-002   | Le Van D     | IT1   |

S2

| <u>Class</u> | Department  |
|--------------|-------------|
| E1           | Electronics |
| E2           | Electronics |
| IT1          | IT          |

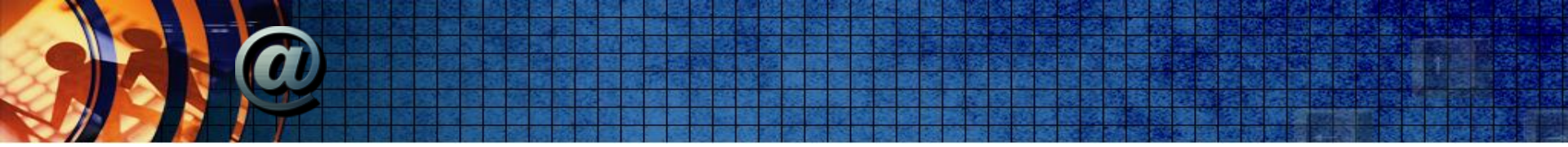
S3

| <u>ID</u> | <u>Subject</u>     | Mark |
|-----------|--------------------|------|
| E1-001    | Electronic Circuit | 8    |
| E1-001    | Digital Technique  | 7    |
| E1-002    | Digital Technique  | 9    |
| E1-002    | Electronic Circuit | 8    |
| E2-001    | Digital Technique  | 6    |
| IT1-001   | Electronic Circuit | 10   |
| IT1-002   | Digital Technique  | 8    |



# Phép tách

- Một phép tách tốt cần phải::
  - Loại bỏ tất cả các bất thường
  - Lưu giữ được các tính chất mong muốn
- Làm thế nào đạt được?
  - **Quá trình chuẩn hóa và các dạng chuẩn**  
(Normalization and Normal Forms)



# Chuẩn hóa dữ liệu



# Khái niệm chuẩn hóa

- Chuẩn hóa: Mô hình quan hệ chỉ chấp nhận duy nhất các quan hệ thoả mãn điều kiện “Mọi giá trị trong quan hệ - nghĩa là tất cả các giá trị của các thuộc tính trong tất cả các bộ - đều phải là nguyên tố (atomic) (nghĩa là không chia nhỏ được nữa)” .
- Nói một cách khác, tại mỗi vị trí cắt nhau của một hàng và một cột trong bảng tồn tại đúng một giá trị và không bao giờ là một tập các giá trị.
- Dạng chuẩn (normal form): Là trạng thái của một quan hệ có được do áp dụng những quy tắc liên quan đến phụ thuộc hàm của quan hệ đó.
- Chuẩn hoá dữ liệu: là quá trình phân rã lược đồ quan hệ chưa chuẩn hoá (có dạng chuẩn thấp) thành các lược đồ quan hệ nhỏ hơn nhưng ở dạng chuẩn cao hơn (có cấu trúc tốt hơn)





# Mục đích của việc chuẩn hóa

- Loại bỏ các dữ liệu dư thừa trong các bảng.
- Xóa bỏ các dị thường trong các thao tác cập nhật dữ liệu.
- Tối ưu hóa cấu trúc của các bảng.



# Các dạng chuẩn

- Dạng chuẩn 1(1 Normal Form - 1NF): Một bảng đạt chuẩn 1 khi và chỉ khi tất cả các cột trong bảng chỉ chứa giá trị nguyên tố, nghĩa là không có nhóm lặp (các cột) bên trong một hàng của bảng.

BEFORE

| S# | PQ |     |
|----|----|-----|
|    | P# | QTY |
| S1 | P1 | 300 |
|    | P2 | 200 |
|    | P3 | 400 |
|    | P4 | 200 |
|    | P5 | 100 |
|    | P6 | 100 |
| S2 | P1 | 300 |
|    | P2 | 400 |
| S3 | P2 | 200 |
| S4 | P2 | 200 |
|    | P4 | 300 |
|    | P5 | 400 |

AFTER

| S# | P# | QTY |
|----|----|-----|
| S1 | P1 | 300 |
| S1 | P2 | 200 |
| S1 | P3 | 400 |
| S1 | P4 | 200 |
| S1 | P5 | 100 |
| S1 | P6 | 100 |
| S2 | P1 | 300 |
| S2 | P2 | 400 |
| S3 | P2 | 200 |
| S4 | P2 | 200 |
| S4 | P4 | 300 |
| S4 | P5 | 400 |

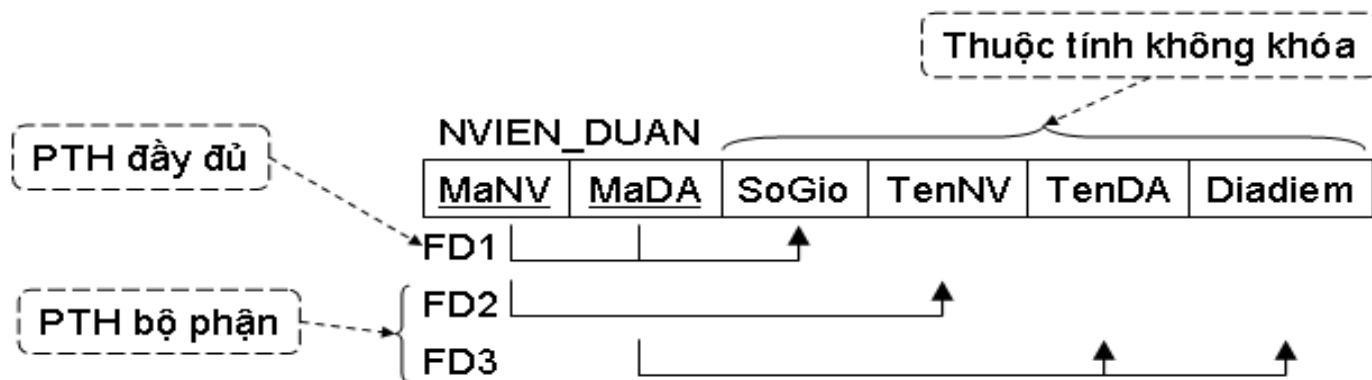
- Kết luận:

- Mọi lược đồ quan hệ đều thuộc dạng chuẩn 1.
- Dạng chuẩn 1 có thể dẫn đến sự trùng lặp dữ liệu. Do đó gây ra các dị thường về cập nhật dữ liệu.



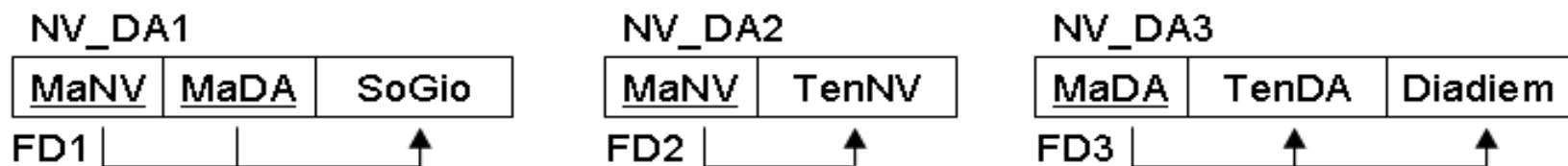
# Các dạng chuẩn

- Dạng chuẩn 2 (2 Normal Form - 2NF): Lược đồ quan hệ R được gọi là thuộc dạng chuẩn 2 nếu nó đã ở dạng chuẩn 1 và **mọi thuộc tính không khóa** của R **phụ thuộc hàm đầy đủ** vào **khóa chính** của R.

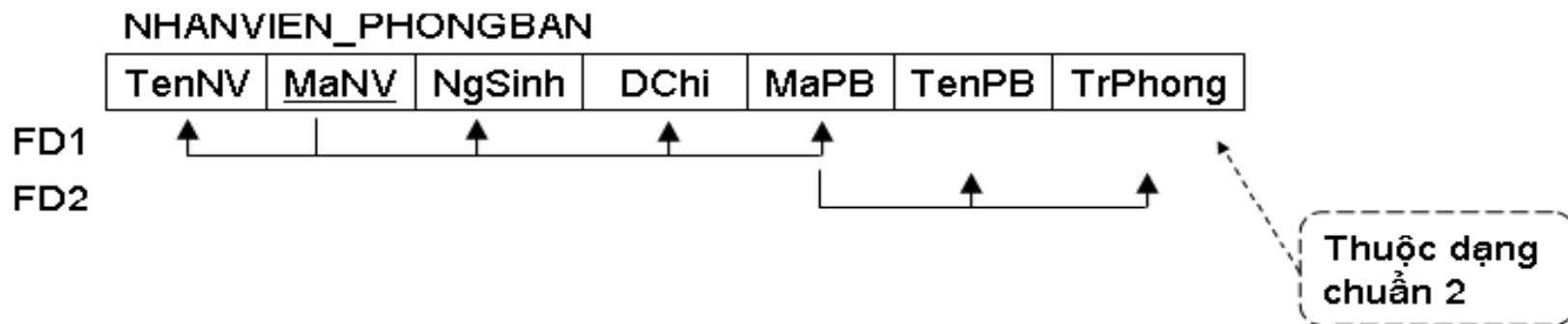




# Các dạng chuẩn



3 lược đồ NV\_DA1, NV\_DA2, NV\_DA3 thuộc dạng chuẩn 2



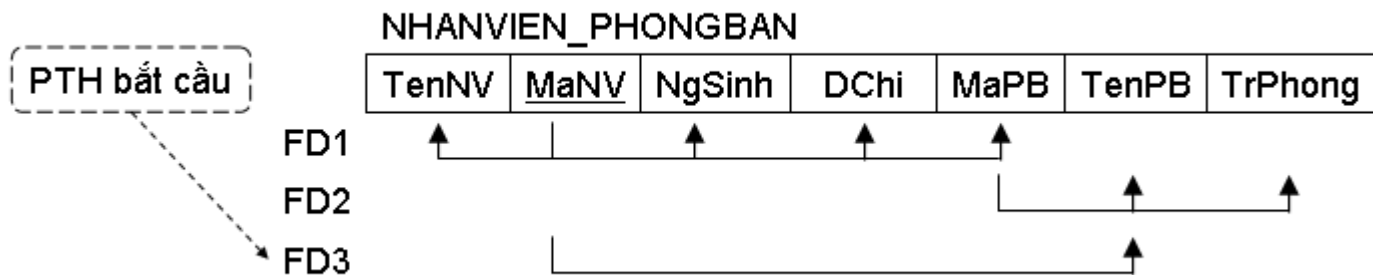
- Kết luận:
  - Mọi lược đồ quan hệ thuộc dạng chuẩn 2 cũng thuộc dạng chuẩn 1.
  - Còn xuất hiện sự trùng lặp dữ liệu. Do đó gây ra các dị thường về cập nhật dữ liệu.



# Các dạng chuẩn

- Dạng chuẩn 3 (3 Normal Form - 3NF): Lược đồ quan hệ R được gọi là thuộc dạng chuẩn 3 nếu:
  - R thuộc dạng chuẩn 2.
  - Mọi thuộc tính không khóa của R không phụ thuộc bắc cầu vào khóa chính của R.

Ví dụ: Lược đồ dưới đây thuộc dạng chuẩn 2 nhưng không thuộc dạng chuẩn 3, vì tồn tại phụ thuộc hàm bắc cầu:

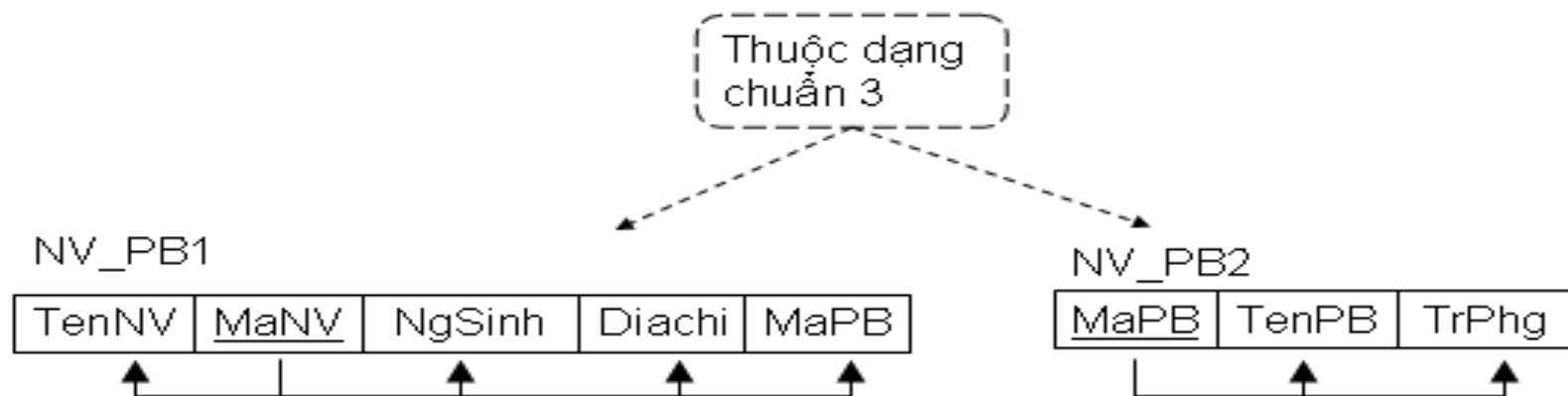


*Rõ ràng:*  $\text{MaNV} \rightarrow \text{MaPB}, \text{MaPB} \rightarrow \text{TenPB} \Rightarrow \text{MaNV} \rightarrow \text{TenPB}$ , Sau khi chuẩn hóa sang dạng 3NF chúng ta nhận được các lược đồ sau:



# @ Các dạng chuẩn

- Kết luận:
  - Mọi lược đồ quan hệ thuộc dạng chuẩn 3 cũng thuộc dạng chuẩn 2.
  - PTH bắc cầu là nguyên nhân dẫn đến trùng lặp dữ liệu.
  - Dạng chuẩn 3 là dạng chuẩn tối thiểu trong thiết kế CSDL





# Các bước đưa về dạng chuẩn 3NF

- Bước 1: Chuyển quan hệ chưa chuẩn hóa thành 1 tập quan hệ tương đương ở dạng 1NF.
  - Cách 1: Loại bỏ các nhóm lặp bằng cách đưa dữ liệu thích hợp vào các cột trống của các hàng chứa dữ liệu lặp
    - Cách này thường được gọi là “làm phẳng” (flattening)
    - Bằng cách này sẽ dư thừa đã được đưa vào quan hệ kết quả, tuy nhiên sự dư thừa này sẽ được loại bỏ trong quá trình chuẩn hóa
  - Cách 2:
    - Chỉ định một hoặc một nhóm các thuộc tính làm khóa chính cho bảng chưa chuẩn hóa
    - Loại bỏ các nhóm lặp bằng cách mang nhóm các thuộc tính lặp lại cùng bản copy của khóa chính sang một quan hệ tách biệt
    - Nhận diện các khóa chính của các quan hệ mới
    - Nếu bảng chưa chuẩn hóa có chưa hơn một nhóm lặp lại hay các nhóm lặp lại lồng nhau thì cách này được lặp đi lặp lại cho đến khi không còn nhóm lặp nữa.



## Các bước đưa về dạng chuẩn 3NF

- Bước 2. Tạo các ánh xạ của quan hệ gốc 1NF để hạn chế mọi phụ thuộc hàm không đầy đủ vào khoá chính để tạo ra một tập các quan hệ 2NF theo các bước sau:
  - Từ quan hệ ban đầu, đối với mỗi thuộc tính của khoá chính, ta tạo một quan hệ tách biệt
  - Tiếp tục tạo các quan hệ tách biệt mới với tất cả các kết hợp của các thuộc tính của khoá chính cho đến khi tạo được quan hệ với toàn bộ khoá chính gốc
  - Chỉ định thuộc tính hoặc kết hợp các thuộc tính này làm khoá chính cho các quan hệ mới
  - Trong quan hệ gốc, tìm sự phụ thuộc hàm đầy đủ của từng thuộc tính trong các thuộc tính còn lại vào các khoá chính mới tìm ra và đặt nó vào quan hệ tương ứng với khoá chính mới này
- Khi đã xét hết các thuộc tính còn lại trong quan hệ gốc thì bỏ đi các quan hệ mà chỉ có các thuộc tính khoá mà không có các thuộc tính không khoá khác



## Các bước đưa về dạng chuẩn 3NF

- Bước 3: Tạo các ánh xạ của các quan hệ 2NF này để hạn chế mọi phụ thuộc hàm bắc cầu của các thuộc tính không khoá vào khoá chính. Kết quả sẽ tạo nên một tập các quan hệ 3NF.
  - Từ quan hệ 2NF ban đầu, mang bản copy của thuộc tính quyết định cùng tất cả các thuộc tính không khoá mà phụ thuộc hàm vào thuộc tính quyết định này sang một quan hệ mới với thuộc tính quyết định sẽ được chỉ định làm khoá chính của quan hệ mới



## Các bước đưa về dạng chuẩn 3NF

- Lưu ý:
  - Trường hợp quan hệ đã ở dạng 3NF nhưng có các khoá ứng cử phủ nhau thì chưa thể khẳng định là đã ở dạng chuẩn BCNF
  - Nếu tồn tại thuộc tính quyết định mà không phải là khoá ứng cử thì phải mang bản copy của thuộc tính quyết định cùng các thuộc tính phụ thuộc hàm đầy đủ vào nó sang một quan hệ mới
- Mục đích chính của quá trình này là giảm được sự dư thừa thông tin, và do đó tránh được các vấn đề dị thường với các phép toán lưu trữ như đã bàn đến ở trên





# Lược đồ khái niệm

Cho cơ sở dữ liệu sau:

|          |           |              |               |               |
|----------|-----------|--------------|---------------|---------------|
| <b>S</b> | <b>S#</b> | <b>SNAME</b> | <b>STATUS</b> | <b>CITY</b>   |
|          | <b>S1</b> | <b>Smith</b> | <b>20</b>     | <b>London</b> |
|          | <b>S2</b> | <b>Jones</b> | <b>10</b>     | <b>Paris</b>  |
|          | <b>S3</b> | <b>Blake</b> | <b>30</b>     | <b>Paris</b>  |
|          | <b>S4</b> | <b>Clark</b> | <b>20</b>     | <b>London</b> |
|          | <b>S5</b> | <b>Adams</b> | <b>30</b>     | <b>Athens</b> |

|          |           |              |              |               |               |
|----------|-----------|--------------|--------------|---------------|---------------|
| <b>P</b> | <b>P#</b> | <b>PNAME</b> | <b>COLOR</b> | <b>WEIGHT</b> | <b>CITY</b>   |
|          | <b>P1</b> | <b>Nut</b>   | <b>Red</b>   | <b>12</b>     | <b>London</b> |
|          | <b>P2</b> | <b>Bolt</b>  | <b>Green</b> | <b>17</b>     | <b>Paris</b>  |
|          | <b>P3</b> | <b>Screw</b> | <b>Blue</b>  | <b>17</b>     | <b>Rome</b>   |
|          | <b>P4</b> | <b>Screw</b> | <b>Red</b>   | <b>14</b>     | <b>London</b> |
|          | <b>P5</b> | <b>Cam</b>   | <b>Blue</b>  | <b>12</b>     | <b>Paris</b>  |
|          | <b>P6</b> | <b>Cog</b>   | <b>Red</b>   | <b>19</b>     | <b>London</b> |

|           |           |           |            |
|-----------|-----------|-----------|------------|
| <b>SP</b> | <b>S#</b> | <b>P#</b> | <b>QTY</b> |
|           | <b>S1</b> | <b>P1</b> | <b>300</b> |
|           | <b>S1</b> | <b>P2</b> | <b>200</b> |
|           | <b>S1</b> | <b>P3</b> | <b>400</b> |
|           | <b>S1</b> | <b>P4</b> | <b>200</b> |
|           | <b>S1</b> | <b>P5</b> | <b>100</b> |
|           | <b>S1</b> | <b>P6</b> | <b>100</b> |
|           | <b>S2</b> | <b>P1</b> | <b>300</b> |
|           | <b>S2</b> | <b>P2</b> | <b>400</b> |
|           | <b>S3</b> | <b>P2</b> | <b>200</b> |
|           | <b>S4</b> | <b>P2</b> | <b>200</b> |



# Lược đồ khái niệm

|                 |                                    |                       |
|-----------------|------------------------------------|-----------------------|
| <u>DOMAIN</u>   | S#                                 | <u>CHARACTER</u> (5)  |
| <u>DOMAIN</u>   | SNAME                              | <u>CHARACTER</u> (20) |
| <u>DOMAIN</u>   | STATUS                             | <u>NUMERIC</u> (3)    |
| <u>DOMAIN</u>   | CITY                               | <u>CHARACTER</u> (5)  |
| <u>DOMAIN</u>   | P#                                 | <u>CHARACTER</u> (6)  |
| <u>DOMAIN</u>   | PNAME                              | <u>CHARACTER</u> (20) |
| <u>DOMAIN</u>   | COLOR                              | <u>CHARACTER</u> (6)  |
| <u>DOMAIN</u>   | WEIGHT                             | <u>NUMERIC</u> (4)    |
| <u>DOMAIN</u>   | QTY                                | <u>NUMERIC</u> (5)    |
| <u>RELATION</u> | S (S#, SNAME, STATUS, CITY)        | <u>KEY</u> (S#)       |
| <u>RELATION</u> | P (P#, PNAME, COLOR, WEIGHT, CITY) | <u>KEY</u> (P#)       |
| <u>RELATION</u> | SP (S#, P#, QTY)                   | <u>KEY</u> (S#, P#)   |



## 5 Các phép toán trên CSDL quan hệ

- Chèn (insert)
- Loại bỏ (delete)
- Thay đổi (change)

Trong mô hình CSDL quan hệ được nêu trên, các phép toán này được áp dụng cho từng bộ của các quan hệ lưu trữ trong máy



# Các phép toán trên CSDL quan hệ

- Phép chèn (INSERT)

- Phép chèn thêm một bộ  $t$  vào quan hệ  $r$   $\{A_1, \dots, A_n\}$  có dạng:

$$r = r \cup t$$

- Ta có thể biểu diễn chi tiết phép chèn như sau:

INSERT ( $r$ ;  $A_1 = d_1, A_2 = d_2, \dots, A_n = d_n$ )

- Trong đó:  $A_i$  với  $i = 1, \dots, n$  là tên các thuộc tính và  $d_i$  với  $d_i \in \text{dom}(A_i)$  là các giá trị thuộc miền trị tương ứng của thuộc tính  $A_i$ .
- Ví dụ, để thêm một bộ  $t_4 = (S_4, \text{Clark}, 20, \text{London})$  vào quan hệ  $S$  ta làm như sau:

INSERT( $S$ ;  $S\# = S_4, SNAME = \text{Clark}, STATUS = 20, CITY = \text{London}$ )



# Các phép toán trên CSDL quan hệ

- Nếu xem thứ tự các trường là cố định, khi đó có thể biểu diễn phép chèn dưới dạng không tường minh như sau:

INSERT (r; d1, d2, ..., dn)

- Mục đích của phép chèn là thêm một bộ vào một quan hệ nhất định
- Kết quả của phép toán này có thể gây ra một số sai sót với những lý do sau đây:
  - Bộ mới cần thêm vào không phù hợp với lược đồ quan hệ đã cho
  - Một số giá trị của một số thuộc tính nằm ngoài miền giá trị của thuộc tính đó
  - Giá trị khoá của bộ mới có thể là giá trị đã có trong quan hệ đang lưu trữ
- Do vậy, tùy từng hệ thống cụ thể sẽ có những cách khắc phục riêng



# @ Các phép toán trên CSDL quan hệ

- Phép loại bỏ (DEL)

- Phép loại bỏ (DEL) là phép xóa một bộ ra khỏi một quan hệ cho trước
- Giống như phép chèn, phép loại bỏ có dạng:

$$r = r - t$$

- Ta có thể biểu diễn chi tiết phép loại bỏ như sau:

$$\mathbf{DEL (r; A1 = d1, A2 = d2, ..., An = dn)}$$

- Hoặc nếu xem thứ tự các trường là cố định thì:

$$\mathbf{DEL (r; d1, d2, ... dn)}$$

- Ví dụ, cần loại bỏ bộ tương ứng với P2, S3 trong quan hệ SP ta làm như sau:

$$\mathbf{DEL (SP; S\# = S3, P\# = P2, QTY = 200)}$$



# Các phép toán trên CSDL quan hệ

- Tuy nhiên, không phải lúc nào phép loại bỏ cũng cần đầy đủ tất cả các loại thông tin của bộ cần loại bỏ
- Nếu có giá trị về bộ đó tại các thuộc tính khoá  $K = \{B1, B2, \dots, Bj\}$  thì phép loại bỏ chỉ cần viết:

**DEL (r; B1 = f1, B2 = f2, ..., Bj = fj)**

- Ví dụ, trong phép loại bỏ nêu trên, vì  $(S\#, P\#)$  là khoá của quan hệ SP nên ta chỉ cần viết:

**DEL (SP; S# = S3, P# = P2)**



# Các phép toán trên CSDL quan hệ

- Phép thay đổi (CHANGE hoặc UPDATE)
  - Trong thực tế không phải lúc nào cũng chỉ dùng phép chèn hoặc loại bỏ đi một bộ mà nhiều khi chỉ cần sửa đổi một số giá trị nào đó tại một số thuộc tính, lúc đó cần thiết phải sử dụng phép thay đổi ("Change" hay viết tắt là CH)
  - Gọi tập  $\{C1, \dots, Cp\} \subseteq \{A1, \dots, An\}$  là tập các thuộc tính mà tại đó các giá trị của bộ cần thay đổi, khi đó phép thay đổi có dạng:

$$r = r - t \cup t'$$

- Hay ta có thể biểu diễn chi tiết phép loại bỏ như sau:

**CH (r; A1 = d1, A2 = d2, ..., An = dn; C1 = e1, C2 = e2, ..., Cp = ep)**



# Các phép toán trên CSDL quan hệ

- Nếu  $K = \{B_1, \dots, B_m\}$  là khoá của quan hệ, khi đó chỉ cần viết:

**CH (r; B1 = f1, B2 = f2, ..., Bm = fm; C1 = e1, C2 = e2, ..., Cp = ep)**

- Ví dụ: Cần thay đổi địa chỉ hãng cung cấp S1 từ London thành Amsterdam trong quan hệ S. Khi đó phép thay đổi có dạng như sau:

**CH (S; S# = S1; CITY = Amsterdam)**

- Cũng có thể không dùng phép thay đổi mà dùng tổ hợp của phép loại bỏ và phép chèn một bộ mới
- Do vậy, sai sót đối với phép thay đổi cũng sẽ xảy ra tương tự như với phép chèn và phép loại bỏ

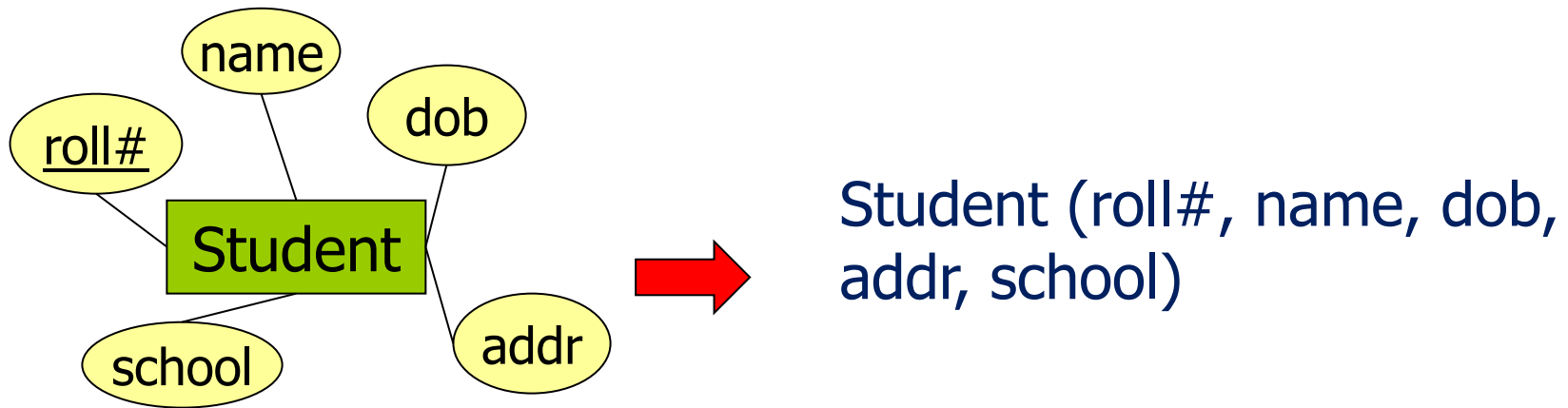


Chuyển từ Mô hình thực thể liên kết  
sang Mô hình quan hệ



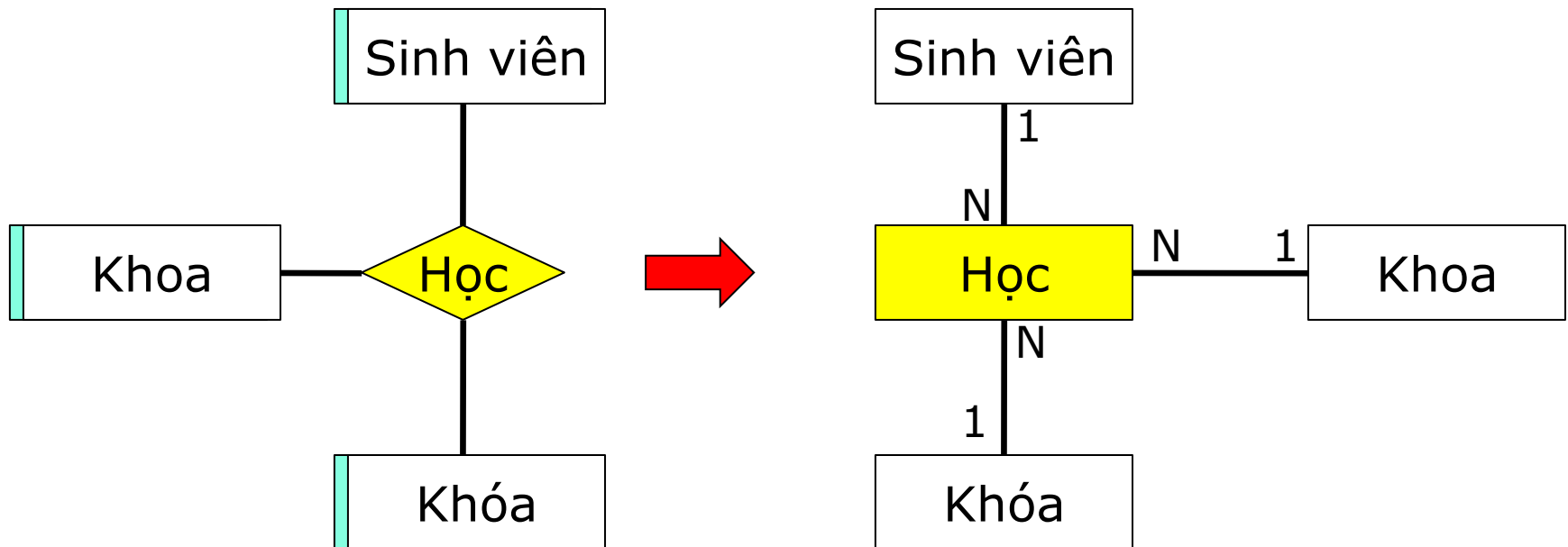
## @ Chuyển từ thực thể sang quan hệ

- Tên thực thể → Tên quan hệ
- Các thuộc tính của thực thể → Các thuộc tính của QH.



## @ Chuyển từ liên kết sang quan hệ

- Chuyển liên kết nhiều ngôi thành các liên kết hai ngôi:
  - Coi liên kết như tập thực thể



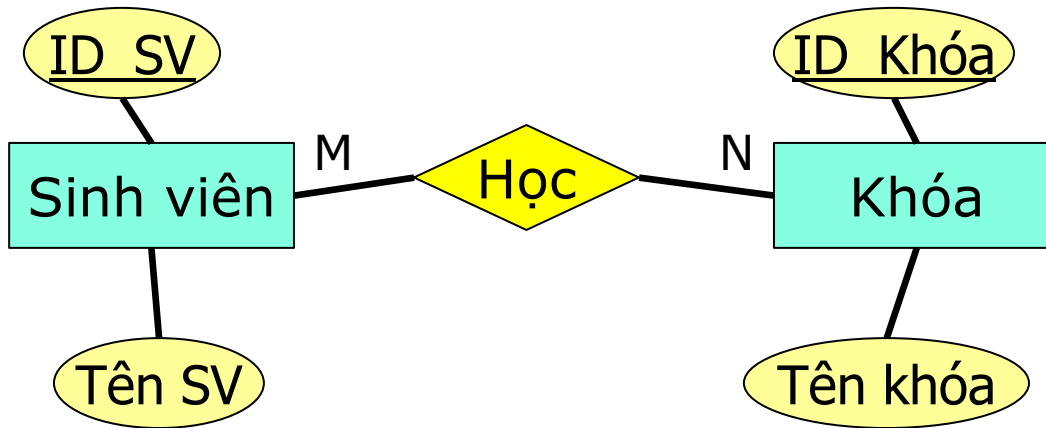


# Chuyển từ liên kết sang quan hệ

- Chuyển liên kết 2 ngôi:
  - Liên kết 1:1: nên ghép 2 thực thể này lại để thành một thực thể chung, rồi chuyển thực thể chung này thành QH
  - Liên kết M:N:
    - Tên liên kết  $\rightarrow$  Tên quan hệ
    - Thuộc tính của quan hệ =  $\cup$  các thuộc tính khóa của các thực thể thành viên.
  - Liên kết 1:N: chỉ cần thêm thuộc tính khóa của thực thể ở đầu 1 vào thực thể ở đầu N, thì liên kết này có thể được bỏ đi.

# @ Chuyển từ liên kết sang quan hệ

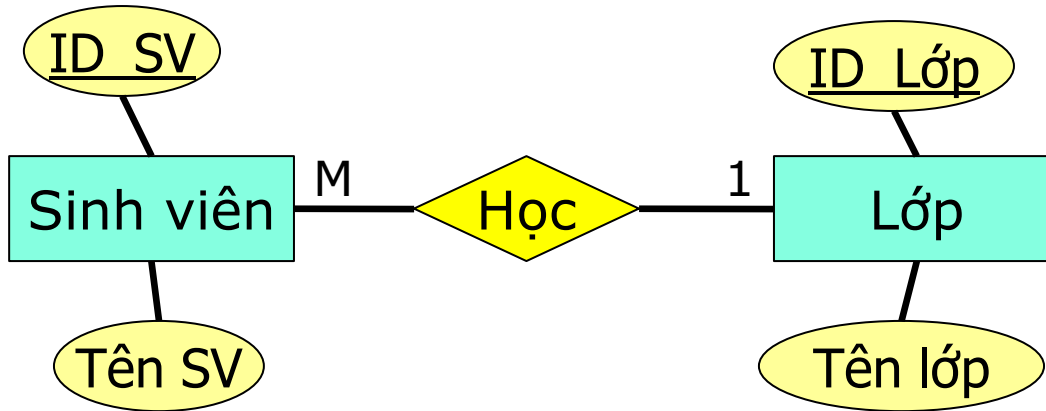
Ví dụ về chuyển liên kết hai ngôi:



Sinh viên (ID SV, Tên SV);

Khóa (ID Khóa, Tên khóa);

Học (ID SV, ID Khóa);



Sinh viên (ID SV, Tên SV,  
*ID\_Lop*);

Lớp (ID Lớp, Tên lớp);



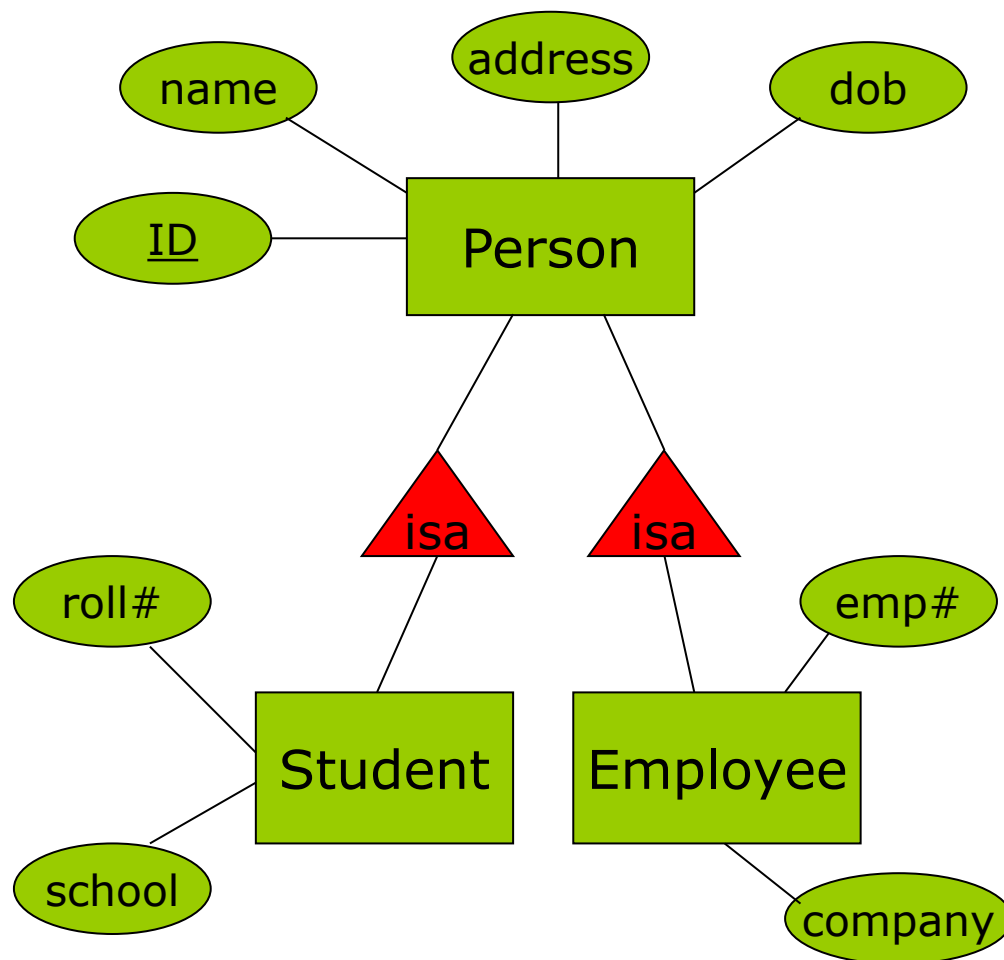
## Chuyển từ liên kết sang quan hệ

- **Chuyển liên kết ISA sang quan hệ:** không cần phải tạo quan hệ mới cho loại liên kết này, nhưng các thực thể tham gia một cây liên kết này cần được xử lý đặc biệt theo 1 trong 3 cách như sau:
  - **Theo góc nhìn Thực thể/Liên kết:** với mỗi thực thể E trong cây liên kết, tạo một quan hệ mà chứa các thuộc tính khóa của gốc và các thuộc tính thuộc về E.
  - **Theo cách tiếp cận hướng đối tượng:** Coi các thực thể như các đối tượng thuộc về một lớp. Với mỗi cây con của cây liên kết, thì tạo ra một quan hệ mà chứa tất cả các thuộc tính của cây con đó.
  - **Sử dụng giá trị rỗng (NULL values):** tạo ra một quan hệ duy nhất bao gồm tất cả các thuộc tính của các thực thể trong cây liên kết.



# @ Chuyển từ liên kết sang quan hệ

- Ví dụ chuyển cây liên kết sau:





## Chuyển từ liên kết sang quan hệ

- Theo góc nhìn Thực thể/liên kết ta được các quan hệ

Person(ID, name, address, dob)

Student(ID, roll#, school)

Employee(ID, emp#, company)



# Chuyển từ liên kết sang quan hệ

## - Theo cách tiếp cận hướng đối tượng :

- Cây liên kết trên có 4 cây con:
  - ✓ Một mình Person
  - ✓ Person và Student
  - ✓ Person và Employee
  - ✓ Cả 3 tập thực thể
- Từ đó ta sẽ có 4 quan hệ:
  - ✓ Person(ID, name, address, dob)
  - ✓ PersonS(ID, name, address, dob, roll#, school)
  - ✓ PersonE(ID, name, address, dob, emp#, company)
  - ✓ PersonSE(ID, name, address, dob, roll#, school , emp#, company)



## Chuyển từ liên kết sang quan hệ

- Theo cách sử dụng giá trị rỗng: Từ cây liên kết, ta tạo ra 1 quan hệ duy nhất:

**Person(ID, name, address, dob, roll#, school , emp#, company))**



# Các quy tắc toàn vẹn dữ liệu

- C.J. Date:
  - Quy tắc Toàn vẹn thực thể (Entity Integrity Rule - EI):
    - Không một thành phần nào của các thuộc tính dùng làm khoá chính trên một bảng dữ liệu được là null
  - Quy tắc Toàn vẹn quy chiếu (Referential Integrity Rule - RI):
    - Nếu một bảng dữ liệu bao gồm một khoá ngoại lai (FK) khớp với một khoá chính (PK) trên một bảng dữ liệu T nào đó, thì mỗi giá trị của FK phải hoặc:
      - bằng trị của khoá chính trên một hàng nào đó của bảng dữ liệu T, hoặc
      - hoàn toàn null (Điều này không nên có vì không kết nối vào đâu)
- Cần tuân thủ các quy tắc toàn vẹn dữ liệu này trong 3 phép toán thao tác CSDL





# Các quy tắc toàn vẹn dữ liệu

- Phép toán Chèn (Insert): Nếu người sử dụng cố thử đưa vào một bộ với khoá ngoại lai không khớp với trị của khoá chính của bảng dữ liệu tương ứng thì để đảm bảo các quy tắc toàn vẹn dữ liệu, ta có thể cho áp dụng một trong 3 nguyên tắc sau đây:
  - Gạt bỏ việc thêm hàng này và yêu cầu người sử dụng đưa vào thông tin của khoá chính trên bảng dữ liệu tương ứng
    - Nguyên tắc này được gọi là nguyên tắc phụ thuộc (DEPENDENT)
  - Tự động chuyển người sử dụng qua một chương trình khác để cho phép người sử dụng thêm vào quan hệ tương ứng một hàng mới với khoá chính cần thiết
    - Nguyên tắc này được gọi là nguyên tắc tự động (AUTOMATIC)
  - Chấp nhận bộ mới với khoá ngoại lai là null, nếu đồng ý chấp nhận null đối với khoá ngoại lai
    - Nguyên tắc này được gọi là nguyên tắc làm rỗng (NULLIFY)



# Các quy tắc toàn vẹn dữ liệu

- Phép toán Thay đổi (Update): Để xác định sẽ xử lý như thế nào khi người sử dụng cố tình thay đổi khoá chính có liên hệ với một khoá ngoại lai, có thể chọn một trong 3 hướng giải quyết sau đây:
  - Tự động thay đổi giá trị của khoá ngoại lai cho khớp với giá trị mới của khoá chính
    - Nguyên tắc này được gọi là nguyên tắc thác đổ hay dây chuyền (CASCADE)
  - Chỉ cho phép thay đổi giá trị khoá chính ở hàng hiện không có giá trị của khoá chính khớp với giá trị hiện có của khoá ngoại lai tương ứng
    - Nguyên tắc này được gọi là nguyên tắc hạn chế (RESTRICT)
  - Cho khoá ngoại lai về null đối với các hàng có giá trị của khoá ngoại lai hiện khớp với giá trị của khoá chính cần thay đổi, nếu đồng ý chấp nhận null đối với khoá ngoại lai
    - Đây là nguyên tắc làm rỗng (NULLIFY)



# Các quy tắc toàn vẹn dữ liệu

- Phép toán Xoá (Delete): Nếu người sử dụng cố tình xoá một hàng chứa giá trị của khoá chính mà ít nhất có một bảng dữ liệu hiện đang sử dụng giá trị này cho khoá ngoại lai thì quy tắc toàn vẹn dữ liệu có thể tăng cường bằng một trong ba nguyên tắc sau đây (cũng tương tự như với phép thay đổi ở trên):
  - Tự động xoá tất cả các hàng trong tất cả các bảng dữ liệu có giá trị của khoá ngoại lai trùng với giá trị của khoá chính cần xoá
    - Nguyên tắc này được gọi là nguyên tắc thác đổ hay dây chuyền (CASCADE)
  - Chỉ cho phép xoá giá trị khoá chính ở hàng hiện không có giá trị của khoá chính khớp với giá trị hiện có của khoá ngoại lai tương ứng
    - Nguyên tắc này được gọi là nguyên tắc hạn chế (RESTRICT)
  - Cho khoá ngoại lai về null đối với các hàng có giá trị của khoá ngoại lai hiện khớp với giá trị của khoá chính cần xoá, nếu đồng ý chấp nhận null đối với khoá ngoại lai
    - Đây là nguyên tắc làm rỗng (NULLIFY)